

LEARNING TO THEORIZE IN MENTAL TERMS

MSc Thesis (*Afstudeerscriptie*)

written by

Gidon Kaminer

under the supervision of **Dr Fausto Carcassi**, and submitted to the
Examinations Board in partial fulfilment of the requirements for the degree of

MSc in Logic

at the *Universiteit van Amsterdam*.

Date of the public defence: Members of the Thesis Committee:

August 28th, 2026

Dr Fausto Carcassi (supervisor)

Dr Malvin Gattinger (chair)

Dr Martha Lewis

Dr Giorgio Sbardolini



INSTITUTE FOR LOGIC, LANGUAGE AND COMPUTATION

Abstract

Contents

1 Development of Theory of Mind	4
1.1 The nature and scope of theory of mind	4
1.2 Developmental milestones (ToM scale)	5
1.2.1 early roots (0-12 months)	5
1.2.2 emergence of epistemic states (12-24 months)	5
1.2.3 “Sally-Anne” False belief task	5
1.2.4 second-order mentalizing	6
1.3 Theoretical perspectives on ToM acquisition	6
1.3.1 Nativism and core knowledge	6
1.3.2 Empiricism and connectionism/associationism	6
1.3.3 (rational) constructivism and the theory theory	6
2 The Bayesian Model	7
2.1 Bayesian Cognitive Modeling	7
2.1.1 Rational constructivism	8
2.1.2 explain Bayesian inference - first generally here and then specifically over programs? or just introduce it there?	8
2.2 The language of thought as a representational medium	8
2.2.1 bayesian inference over structured representations	8
2.2.2 why programs for theory of mind?	9
2.3 Bayesian program induction as the learning engine	9
2.4 Hierarchical Bayesian models	10
2.5 The DreamCoder algorithm	11
2.5.1 Wake: exploration/enumeration/search (Bayesian inference)	11
2.5.2 Sleep: Abstraction (library learning, compression)	11
2.5.3 Sleep: dreaming (amortized inference)	11
2.6 Learning theory of mind via inverse planning	11
3 Implementation	13
3.1 Objective	13
3.2 Task Corpus	13
3.2.1 necessity filters	15
3.3 interpreter	16
3.4 program representation and type system	17
3.5 Primitives	18
3.5.1 core primitives	18
3.5.2 atomic (phase 1)	18
3.5.3 decomposed (phase 2)	18
3.5.4 arity (phase 3)	18
3.6 ECD architecture	18
3.6.1 Enumeration	18

3.6.2	Compression	20
3.6.3	Dreaming	22
4	Results and Discussion	23
4.1	Found abstractions	23
4.1.1	atomic primitives	23
4.1.2	decomposed primitives	24
4.2	The structure of theory of mind	24
4.3	Benefits of concept learning	25
4.3.1	Per-variant belief coverage + combo	28
4.3.2	Behavioral probe: Passing the false-belief test	29
4.4	Ablation experiments	31
4.4.1	Silo vs joint stitch	31
4.4.2	Corpus-mix dose-response	31
4.4.3	Budget dose-response	31
4.4.4	Dreaming ablation	31
4.4.5	curriculum ablation	31
4.4.6	Seed robustness	31
4.5	Discussion and rebuttals	31
5	Further work	34
5.1	Comparison to LLMs	34
5.2	Polymorphism	34
5.3	task sets	34
5.4	persistent models and interaction	35
5.5	breaking down optimize	35
5.6	probabilistic primitives	35
5.7	vs. real world	35
6	Appendix	36
	Bibliography	37

Chapter 1

Development of Theory of Mind

1.1 The nature and scope of theory of mind

The ice cream peddler's scratchy jingle got louder still as he brought his cart to a stop nearby. A child, hitherto engrossed in sandbox amusements, abandoned her playmates abruptly. She darted across the playground, hurtling at full speed towards a gaggle of parents. Absorbing the impact of a 20 kg cruise missile, the target continued his conversation unfazed by the girl now tugging at his jeans. Only when he deigned to look down at the child did he shriek and go pale. But the girl barely had time to react, because just then she heard her name shouted from the other side of the playground. She turned to its source and ran towards a similar-looking man whose outstretched arms held two ice cream cones.

You chuckle at the scene, observing from a nearby park bench. The girl, wanting ice cream, pestered a man she thought was her father. The man, undoubtedly accustomed to his own nagging kids, ignored the girl thinking it was his own child. Descriptions like these, in terms of desires, beliefs, and intentions, seem trivial and obvious. But there's something miraculous about them. How are you able to peer into other peoples' minds? Mental content is unobservable, all you see is a sequence of physical actions. And yet humans, even infants, are experts at inferring mental content rapidly and accurately.

Humans have the cognitive ability to impute mental states to others, to attribute a belief, desire, or intention to another person. This capacity is known as "theory of mind", "folk psychology", the ability to "mindread", to "mentalize". It is integral to everything that makes us human: to cooperate with someone you need to understand what they think, what they want. To decode someone's words, you need to begin with what they're trying to (by the Rational Speech Act framework). To understand social roles like "helping" and "hindering", you need to attribute an intention to them. Humans make these inferences so naturally that it seems like not much of an impressive ability at all, and "mindreading" seems too fabulous a term for something so mundane. But the ubiquity of these inferences should make the human capacity for mentalizing even more impressive, not less.

In this thesis we study the question: how is this capacity acquired? How do humans, even at a young age, come to acquire this ability? We'll begin by

in this section:

- **the problem of other minds** framed as a fundamental challenge: minds are unobservable, yet humans (even infants) are experts at inferring their contents rapidly and accurately
- **function of ToM** its role in social interaction, such as cooperation, communication (pragmatics, RSA), and understanding social roles like "helping" or "hindering"

"Theory of Mind (ToM) refers to the cognitive capacity to attribute mental states to self or others." Other names for the same capacity include:

- “commonsense psychology”
- “naive psychology”
- “folk psychology”
- “mindreading”
- “mentalizing”

[MSS12]

mental states include things like

- perception
- bodily feelings
- emotional states
- propositional attitudes (beliefs, desires, hopes, intentions)

[MSS12]

We’ll be focusing on the attribution of propositional attitudes.

1.2 Developmental milestones (ToM scale)

1.2.1 early roots (0-12 months)

discuss evidence that infants as young as 6 months old interpret motion as goal-directed and apply a “teleological stance”, an intuitive schema relating actions, goals, and environmental constraints (this also helps later to defend **optimize**) as non-intensional

ToM scale: Diverse desires [Repacholi and Gopnik 1997 “Early reasoning about desires: evidence from 14 and 18 month olds”] 18 month olds can identify diverse desires and reason about them

Woodward 1998: attribution of object-directed goal in 5-6 month olds. Infants already see agentically

teleological stance [Gergely, G., Nádasdy, Z., Csibra, G., & Biró, S. (1995). Taking the intentional stance at 12 months of age. *Cognition*, 56, 165–193]

1.2.2 emergence of epistemic states (12-24 months)

review findings on infants’ sensitivity to perceptual access (e.g. knowing an agent can only have a goal for an object they can see)

ToM scale: Knowledge access

1.2.3 “Sally-Anne” False belief task

milestone where children transition from reality-centric responses to representing false beliefs around age 4

ToM scale: False beliefs

Description of the false belief task [WP]. Results: 40% of the 4-year olds and 90% of the 6 to 7 year olds correctly predicted Maxi would look at the location where he believes the chocolate is, rather than its actual location. When information access or lack thereof was made explicitly salient, it rose to 50% of 4-year olds. But children under 3.5 had quite low rates of solving, and their

performance didn't improve when the scenario was made more explicit. So the conclusion is that around 3 or 4 years old, children achieve the ability to attribute minds.

1.2.4 second-order mentalizing

brief overview of more complex social reasoning that emerges later (e.g. "I think that you think"), where the representations have to be richer, the model has to be higher-dimensional

1.3 Theoretical perspectives on ToM acquisition

1.3.1 Nativism and core knowledge

view that ToM is a built-in module, a core system [Leslie]. Discuss the "poverty of stimulus" argument, the data is too sparse for general learning. Which implies innate domain-specific constraints like the "principle of efficiency"

[Berwick 2011] for poverty of stimulus, and Chomsky 1971

"On one side, a purely nativist response was to deny learning and to focus on characterizing the detailed representations that were already in place" [GCT24]

1.3.2 Empiricism and connectionism/associationism

view that ToM is learned through statistical patterns and bottom-up associations from experience.

the structure exists, it's just not in the data. So how can a learner infer abstract concepts like "belief" if they aren't in the data?

[McClelland 2010] on emergence

[McClelland & Rumelhart, 1986] and [Rogers & McClelland, 2004] on connectionism

statistical models of learning defined over large numerical vectors [Ten+11], [GCT24]

"On another side, an empiricist response was to suggest that structured representations were not necessary, that learning (and the inferences that followed) was simply a bottom-up process of learning statistical associations" [GCT24]

1.3.3 (rational) constructivism and the theory theory

Introduce the "child as scientist" metaphor. children don't just associate, they construct causal models, intuitive/folk theories of the mind that they revise in light of new evidence, similar to scientific progress.

developmental (Piagetian) account of building up from smaller components

rational constructivism (reconstructing constructivism) adopts the constructivist idea of theory change but seeks a more formal mechanism than classic Piagetian accounts.

I'm gonna end the section by noting that while the theory theory provides a powerful metaphor, it lacks a precise computational backbone to explain how children search through the infinite space of possible theories. Basically that the strength of the nativist argument lies in part in the supposed lack of algorithms for learning those structured representations. then this sets up chapter 2 to introduce bayesian program induction (via DreamCoder) as that formal implementation.

Chapter 2

The Bayesian Model

2.1 Bayesian Cognitive Modeling

- Marr's levels of analysis
- computational: what is the thing that the system is doing
- it's doing bayesian updating. it's finding the simplest theory.
- algorithmic: the procedure by which the solution is produced
-

identify the computational problem that the mind is trying to solve: the problem is find the best explanation. we argue that best is determined logically by bayes' rule. so it's doing bayesian updating. then the algorithmic level is the procedure at which it's produced, ECD

engineering: building AI, building algorithms that solve computational problems

reverse-engineering: identify the abstract computational problems that the mind must solve to do what it does. knowing that the mind solves these certain problems, what are the AI systems we have that can solve problems like that. Then we can model the mind as that kind of system.

Start by thinking about the computational problem that the mind is solving. out of all the data, its task is to figure out the underlying data generation process, i.e. what's going on in the world. this is the problem of positing hypotheses, assigning prior probabilities to them, and updating the probabilities based on new data. Then Bayes' rule shows us the ideal solution to the problem. Then if we see that humans perform similarly to these algorithms, we have better evidence that it's this kind of problem solving [GCT24]

bayesian models focus on the computational level, the logic of the problem, what the information processing system is doing.

[Marr 1982]

Bayesian framework addresses the question: "how to update beliefs and make inferences in light of observed data". Starts with the logic of the inference when generalizing, i.e. what's happening computationally, rather than how it does it algorithmically. [PTR11]

- computational level: "characterizes the problem that a cognitive system solves and the principles by which its solution can be computed from the available inputs in natural environments"
- algorithmic level: "describes the procedures executed to produce this solution and the representations or data structures over which the algorithms operate"
- implementation level: "specifies how these algorithms and data structures are instantiated in the circuits of a brain or machine" [Ten+11]

reverse engineering means start at the computational level, understand what the system is doing, then get into the algorithm. So it's top-down, or function-first. [GCT24]

rational analysis framework. analyze cognition in terms of adaptive solutions to environmental problems [Anderson 1990]

2.1.1 Rational constructivism

middle way between nativism and empiricism, where domain-general statistical mechanisms construct domain-specific structural knowledge

“how do our minds get so much from so little?” [GCT24]

how children’s minds grow [Gopnik & Meltzoff, 1997]

Bayesian cognitive models explain how people learn with abstract structured knowledge [Gopnik & Tenenbaum, 2007; Griffiths, Chater, Kemp, Perfors, & Tenenbaum, 2010; Perfors, Tenenbaum, & Wonnacott, 2010; Griffiths, Sobel, Tenenbaum, & Gopnik, 2011b; Xu, 2019; Spelke, 2022]

unifying the two [GCT24], moving beyond dichotomies:

- empiricism versus nativism
- domain-general versus domain-specific
- logic versus probability
- symbols versus statistics

“How can domain-general mechanisms of learning and representation build domain-specific systems of knowledge? How can structured symbolic knowledge be acquired through statistical learning?” [Ten+11]

Powerful abstractions can be learned surprisingly quickly, together with or prior to learning the more concrete knowledge they constrain. Structured symbolic representations need not be rigid, static, hard-wired, or brittle. Embedded in a probabilistic framework, they can grow dynamically and robustly in response to the sparse, noisy data of experience. [Ten+11]

2.1.2 explain Bayesian inference - first generally here and then specifically over programs? or just introduce it there?

2.2 The language of thought as a representational medium

2.2.1 bayesian inference over structured representations

contrast the symbolic approach (structured, compositional) with the connectionist approach (unstructured, associative weights), arguing that programs capture the productivity and open-endedness of human thought

concepts can be embedded and composed etc.

defining probabilities over structured symbolic forms of representation like graphs, grammars, predicate logic, relational schemas, and functional programs [GCT24]

only programs capture full breadth and depth of people’s complex capabilities to understand and execute algorithms [Goodman et al 2015]

with code we can model both procedural and declarative knowledge using a single format of representation [Rul]

Lambda calculus as formal language for compositional semantics [Heim & Kratzer, 1998] [Steedman, 2000]

and for other learning tasks [Piantadosi, Goodman, Ellis, & Tenenbaum, 2008] [Liang, Jordan, & Klein, 2009, 2010] [Zettlemoyer & Collins, 2005, 2007]

2.2.2 why programs for theory of mind?

social reasoning requires recursive, compositional representations to handle nested mental states. There's no cleaner way of doing this than with programs. it's basically already a program.

learning is programming. learning a concept is building a program composed of lower level primitives, building an algorithm that does something. symbolic programs are the best formal knowledge representation [Rul]

embedded queries for choices of another agent, intuitive psychology [Stuhlmüller & Goodman, 2013] [Goodman 2015]

2.3 Bayesian program induction as the learning engine

Prior vs. Likelihood: Introduce Bayes' Rule as the engine of learning, where the learner trades off the simplicity of a program (the prior) against its fit to observed behavior (the likelihood)

Piantadosi 2012:

- likelihood function that uses the size principle, penalize overly broad hypotheses [Tenenbaum 1999]
 - used for cross situational word learning [Frank, Goodman, and Tenenbaum (2007)]
 - used for solving subset problem in compositional semantics [Piantadosi et al. (2008)]
- prior is from rational rules model [Tenenbaum, Feldman, and Griffiths (2008)] which first linked probabilistic inference with formal, compositional, representations
 - assumes learners prefer simplicity

likelihood is: data have zero probability if they disagree with classification rule, and constant probability otherwise [Goodman 2008]

Bayesian inference: rational framework for updating beliefs given observed data [Jaynes, 2003; Mackay, 2003]

background knowledge is constrained hypothesis space, finer-grained knowledge is the prior degrees of belief in the hypotheses [Griffiths 2024]

The likelihood reflects the fit between hypothesis and data, and the prior indicates the a priori plausibility of a hypothesis (which might decrease for hypotheses that have low frequency, are complicated, or seem otherwise improbable). The contribution of these two factors to the conclusions that we should draw is fairly natural and makes intuitive sense in a variety of contexts. Returning to an example introduced in chapter 1, if you see John coughing (your data d), you might consider three hypotheses about the cause of the cough: a cold (h_1), lung disease (h_2), or heartburn (h_3). You might rule out heartburn on the basis of fit, since it might only slightly increase the chance of coughing. A cold and lung disease both fit well with the cough—they increase the probability of coughing—but they differ in plausibility. Normally, a cold is far more common than lung disease, and thus might be the hypothesis that you would select to explain the coughing. However, the plausibility

of these two hypotheses might change if you were passing by a hospital and saw John coughing inside. All inductive inferences require considering fit and plausibility. Bayes' rule just tells you how they should be combined to reach a conclusion, using the common language of probability theory to determine the impact of each factor. In the context of cognitive science, priors become a useful way of describing the inductive [Griffiths 2004]

Without the constraints imposed by the prior, no meaningful generalizations would be possible. Without the likelihood, nothing could be learned from multiple examples beyond simply eliminating inconsistent hypotheses. ... The prior determines which concepts count as "natural," whereas the likelihood generates the specificity preference and determines how the strength of that preference—and thus the sharpness of generalization—increases as a function of the number of examples. [Xu 2007]

2.4 Hierarchical Bayesian models

Blessing of abstraction, overhypotheses, HBMs

Explain how learning overhypotheses enables a learner to acquire the form of a theory (i.e. agents are rational) before learning specific goals

abstract knowledge can be acquired faster than specific facts, by pooling evidence across multiple scenarios. This could explain how children acquire core social constraints early in life.

theory of mind is the overhypothesis that structures instances

early Bayesian models addressed only the computational level, just stated what the system can do (optimal statistical inference), without specifying how it does it. HBMs get closer to the algorithmic level, showing "cognition as approximately optimal inference in probabilistic models defined over a learner's subjective and dynamically growing mental representations of the world's structure, rather than some objective and fixed world statistics." [Ten+11]

e.g. concept learning is set membership and causal reasoning is directed graph relations. but it's computationally prohibitive to list all the possible hypotheses, their priors and likelihoods, it's combinatorial. So there's higher level constraints like a bipartite graph or something (the mediocl example) [Ten+11]

General reference for HBMs, assumptions at multiple levels of abstraction [Gelman, Carlin, Stern, & Rubin, 1995]

ideal learner of abstract knowledge is one that learns over a hierarchy of models [Tenenbaum, Griffiths, & Kemp, 2006]

The blessing of abstraction: Learning high level abstractions can be faster than learning low-level instances [KPT07]

HBMs for domain-specific abstract causal knowledge [KPT07]

HBMs for simple relational theories [KT08]

a learner who is simultaneously learning abstract and specific knowledge is almost as efficient as a learner with an innate (i.e. fixed) and correct abstract theory [GUT11]

HBMs for domain general causality learning [GUT11]

“hypothesis spaces of hypothesis spaces, with priors on priors” [GCT24] So over a longer timescale you can learn the priors needed for a specific learning task.

The overhypotheses constrain the space of lower-level hypotheses

[KT08] gives an example of how HBMs can be used to learn overhypotheses about feature-variability (e.g. the shape bias) which help to do categorization (the lower-level learning task)

[Ten+11] gives an example of HBMs for constraining directed graph models, which are then used for explaining observed events. First learn intervention-based causality, then use that to constrain inferences about instances of causality.

introduce the idea that learning is bootstrapped and “gets off the ground” by combining strong, potentially innate, domain-general mechanisms with minimal, skeletal domain-specific knowledge

Minimal nativism: “strong but domain-general inference and representational resources are aided by weaker, domain-specific perceptual input analyzers” [GUT11]

first the most abstract domain knowledge comes into place and then the specific knowledge [Wellman and Gelman 1998]

[GUT11]

blessing of abstraction also defined in [PTR11]

2.5 The DreamCoder algorithm

2.5.1 Wake: exploration/enumeration/search (Bayesian inference)

The wake phase solves the inverse problem: searching for the program (intention) that most likely generated the data (actions)

[Ellis 2020] [Palmarini 2024]

2.5.2 Sleep: Abstraction (library learning, compression)

Explain how refactoring and compression automate the discovery of new social primitives, implementing the Theory Theory by allowing the learner to grow their own language of thought

[Palmarini 2024] [Ellis 2020]

2.5.3 Sleep: dreaming (amortized inference)

explain the neural recognition network as a way to “compile” slow, deliberative Bayesian search into fast, “intuitive” social reasoning.

2.6 Learning theory of mind via inverse planning

agents as approximately rational actors, inverse planning

defining the generative model. agents move in an environment (MDP/POMDP) to maximize utility efficiency

Bayesian inverse planning [Baker, Saxe, and Tenenbaum (2009)]

“naive utility calculus” [Jara-Ettinger, Gweon, Tenenbaum, and Schulz (2015)]

Assume that the observer sees the agent as a rational planner. Then planning computation is solution to markov decision process.

- input: utility and belief functions defined over agent's state space and state-action transition functions
- output: series of actions the agent should perform to maximize utility or fulfill goals [Lak+16]

then the observer can simulate what the agent would do if the agent had certain goals/policies etc, and thus infer those from what actually happens by comparing the simulation to reality. [Lak+16] analogy to physics simulations. Say I want to figure out

First we have principle of rationality, expectation that agents will plan rationally given their world model. Then we reverse-engineer what their mental states are that caused their behavior, based on that principle. Formalizing Dennett's intentional stance. [BST09]

Explanation by rationalization, Bayesian theory of mind framework [Baker 2012]

model causal relation between beliefs, goals, actions as rational probabilistic planning in Markov decision problem. [Baker 2012]

observer uses Bayesian induction to work backward from a trajectory to the most likely goals and beliefs

humans can easily observe agents' actions and then infer the beliefs, desires and intentions that led to those actions, assuming that the agent is approximately rational and goal directed

[Baker, Saxe, and Tenenbaum (2009)], [Lucas, Griffiths, et al. (2014)], [Jern, Lucas, and Kemp (2017)], [Jara-Ettinger, Gweon, Schulz, and Tenenbaum (2016)] [Baker, Jara-Ettinger, Saxe, and Tenenbaum (2017)] [Ullman et al. (2009)]

Forward-planning/rational decisionmaking: given an agent's intentions etc, predict their future actions

inverse planning: given an agent's actions, what were their intentions/beliefs/world model?

use Bayes rule to produce posterior over possible mental states

Baker et al 2017

Chapter 3

Implementation

3.1 Objective

Modularity-nativists argue for the existence of a Theory of Mind Module (ToMM), a natively-endowed domain-specific system for understanding the behavior of intentional agents. In Alan Leslie’s characterization the ToMM introduces attitude concepts like *Believes* and uses a “proprietary representational system” for ascribing attitudes to agents [Les94]. Our goal is to demonstrate that the ability to theorize in mental terms can in principle be *learned*, without the need for a native ToMM.

We initialize the system with a library of domain-general primitives, none of which posit an agent’s intentional state, nor encodes attitude concepts like *believes*. Lacking bespoke ToM capacities, such a system would on Leslie’s view not be able to reason about intentional agents, to attribute a belief to an agent, to explain an agent’s movement as navigation according to a private intensional representation. We show that the system learns *useful abstractions* of domain-general functions, abstractions that encode attitude concepts like *Believes* and attribute an intensional state to an agent.

In our first trial (phase 1) we show that, given the domain-general capacity to represent counterfactual world states, a system can learn a functional abstraction that encodes a notion of belief attribution. The learned function attributes a counterfactual world state to a particular agent, determines what the agent’s movement would be based on its own private (non-world) representation, and then renders only the agent’s movement as a real-world artifact (so it doesn’t render the agent’s believed world as the real world). So this use of the capacity for counterfactual representation is discovered in program space.

In the second trial (phase 2), we show that the system can arrive at an abstracted notion of belief starting from an even more basic language of 2-ary combinators. We decompose the counterfactual representation machinery, initializing the system with a library of product-category combinators that are unambiguously domain-general. We show that enumeration recovers the basic structure of belief attribution even from these atomized primitive operations.

3.2 Task Corpus

We created a new corpus consisting of various grid manipulation tasks. Most of the tasks can be easily solved using a purely extensional explanation, but some can only be (efficiently) explained by positing intentionality, by attributing a belief to an agent (i.e. by associating an intensional grid representation to an agent). Inspired by Piaget’s theory of stages of cognitive development [CITE] and Wellman’s theory of mind scale [CITE],

A *trajectory task* is an $s \times s \times n$ matrix A that depicts a visual scene occurring over n time steps, where each frame is a *grid* of width and height s (we denote the i th grid as A_i). The solver

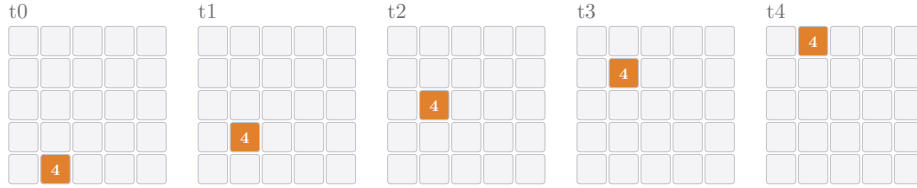
just receives the initial grid A_0 , the first frame (an $s \times s$ grid) of task A . Its goal is to find a program that generates the full A matrix, i.e. a program that posits the underlying data generation process. A correct solution is a function f that, given a grid A_i where $i < n$, returns the next grid in the sequence $f(A_i) = A_{i+1}$.

Tasks are classified by task type. For example, one type of trajectory task is a *movement* task, which depicts a one-cell value moving steadily in some direction for a certain duration. An instance of a movement task might have the value 4 moving one step up in each successive grid. A solution to that task might be a program that encodes “value 4 moves up”.

Physics — a body moves [physics]

(step v d)

val=4, dir=up

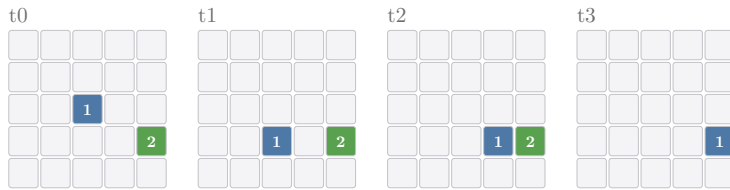


Another type of trajectory task is a *desire* task, which depicts a value navigating towards a goal. The below instance depicts value 1 navigating towards goal 2. A solution to the task might be a program that encodes “value 1 navigates towards value 2”

Desire — seek a goal (true belief) [desire]

(optimize (neg_dist gv) av)

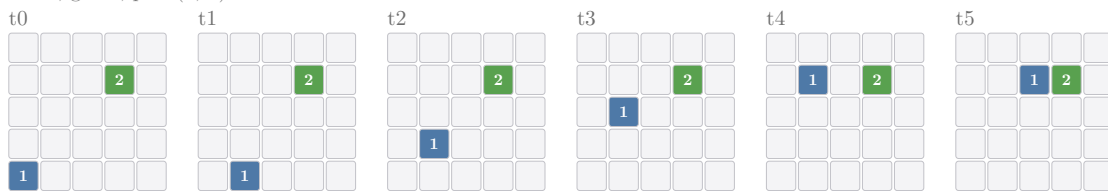
av=1, gv=2



False belief — detour round an invisible wall [belief_wall]

(fork (compose (wall_at r c) (optimize (neg_dist gv) av)) (sync_to_world av))

av=1, gv=2, pw=(3, 0)



The solver is an observer trying to make sense of a scene, just like a human subject in a psychology experiment. The solver has access to the entire task’s history, it just needs to explain it. The enumerator posits an explanation, it’s checked against the ground truth to see if it correctly produces it, and then the enumerator tries again if it doesn’t.

Whereas a trajectory task’s input is one initial grid a_0 , a *template task*’s input is a pair of grids: a *working* grid and a *template* grid, both of dimensions $s \times s$. Given $(working, template)$, the task is to produce $working'$, a version of the working grid modified according to the pattern given in the template grid. This format is loosely inspired by the ARC-AGI-1 task corpus [CITE].

Our corpus consists of various kinds of tasks. Most tasks can be solved without belief attribution, but some tasks require belief attribution.

require belief attribution:

- wall false belief: goal-oriented motion, navigating based on a believed grid which has a wall where one doesn't exist
- witness false belief: two agents, one has a false belief that a wall is there and navigates to avoid it, the other has a true belief about the world so doesn't avoid the wall
- goal-displacement false belief: classic sally-anne,
- false-obstacle belief
- dual belief

don't require belief attribution:

- simple movement
- desire: goal-oriented motion (uses `optimize`)
- overlay: moving object leaves a trail (uses `fork`)
- comet: goal-oriented motion with moving trail (uses `fork`)
- flee: like desire but with maximizing distance
- deletion: grid edit remove one cell
- denoise: grid edit remove a value
- underlay: complement to overlay
- obstacle: goal seeker detours. the grid doesn't have an obstacle there, but the goal seeker detours nonetheless. So the best explanation is `compose( wall_at r c, optimize(neg_dist gv) av )`, that the agent actually is navigating by a real wall that is there, and it's just that I the viewer don't see it
- relocation: move a wall from one location to another
- registration: snap one object onto the template
- perception: registration complement, record a sensation into the private channel
- multi-registration: snap all objects
- registration-except:
-

3.2.1 necessity filters

one of the challenges of creating such a corpus is making tasks that require belief attribution.

checks in the (belief) task generators to make sure they aren't physically explainable, so they're not underdetermined. For example, in a wall false belief task an agent might have a false belief about a wall being there, but at a location nowhere near the agent's path to its goal. So the agent can still navigate to the goal freely as if it didn't have such a false belief, positing the false belief isn't necessary or helpful to explain the scene. So those scenes are rejected.

we implement a filter that

3.3 interpreter

Say we have a candidate program ρ , so the enumerator needs to check for each task x in the corpus whether ρ solves task x . A program ρ solves task x iff running ρ on initial frame x_0 outputs the entire history x . So every solution should be a program that takes as input a $d \times d$ frame and returns a $d \times d \times n$ stack of frames, a program of the form: “iteratively apply transition function f , starting with initial frame x_0 , for n timesteps”. So every solution program has the same outer structure, with only f varying between programs.

This presents a problem: this shared structure, present in every program, will be baked-in to every abstraction. For example say ten tasks in a corpus depict some object falling down, so stitch should identify (`step down #0`) as a shared structure and add this “gravity” primitive to the library. But because “apply f , starting with x_0 , for n timesteps” is a structure common to all programs, stitch would add an overspecified abstraction $f_1 := \lambda \theta \ 1. \text{iteratively apply } (\text{step down } \#0), \text{ starting with } x_0, \text{ for } \#1 \text{ timesteps}$ rather than the more general $f_2 := \lambda \theta. (\text{step down } \#0)$.

The problem is that f_1 doesn’t compose, which defeats the purpose of library learning. Say we abstracted f_1 as “gravity”, and we now want to use it to solve a task involving two objects both dropping to the ground. We can’t do that, because f_1 ’s output is of type `matrix`, an entire 3d scene depicting just one falling object. But if we’ve got an f_2 abstraction with output type `fn`, we can create a program like (`compose (fn_2 4) (fn_2 5)`) that reuses the abstraction for each component of a scene within that same scene (where `compose` is of type `fn -> fn -> fn`).

Since “iteratively apply transition function f , starting with initial frame x_0 , for n timesteps” is common to every program, what the enumerator is looking for is really just that transition function f . The rest is just what’s needed to render a scene given a transition function, to check whether f correctly produces a given task scene. So we simply move the rendering machinery from program space to the evaluation framework.

We implement an interpreter that, given a candidate program (transition function) f , a task’s initial frame x_0 , and the task’s duration (z-dimension) n , generates a scene x' by applying f iteratively (starting with x_0) for n timesteps.

```
def unfold(g: grid, T: int, f: fn) -> mat:
    frames = [g.copy()]
    for _ in range(T - 1):
        g = f(g)
        frames.append(g.copy())
    return np.stack(frames)
```

The interpreter is minimal by design. At each timestep it just applies f to the previous grid to generate the new grid, and appends the new grid to the list of grids. So `unfold`’s state is just a single grid: it doesn’t encode a world/model grid pair, nor a registry of agents and goals.

A transition function f is a grid endomorphism `fn :: grid -> grid`

There is nothing representational about the interpreter, since it only builds up one sequence of grids. The ability to create multiple representations of the world, to create a representation that

deviates from the real world (and attribute it to a particular agent), is not encoded in the interpreter. So to assume the interpreter as innate to the system does not smuggle in a theory of mind.

for some of the tasks, we use an interpreter for programs of root `fn_p_g :: (grid, grid) -> grid` since those tasks don't involve just one input grid but a pair. That uses `unfold_with_template(g, template, T, c)`

```
def unfold_with_template(g, template, T, c):
    """grid, grid, int, fn_p_g -> mat: thread g; each frame pair it with a
    *constant external* template and apply commit c. Unlike fork, the second
    channel is a given input, not a privately derived model – so a program that
    uses sync_to_world here is doing registration, not holding a belief.
    """
    frames = [g.copy()]
    for _ in range(T - 1):
        g = c((g, template))
        frames.append(g.copy())
    return np.stack(frames)
```

3.4 program representation and type system

Every program, primitive, and invented abstraction is an instance of the `Delta` class, i.e. it is an object that can be a node in an expression tree. A `Delta` is initialized with a `head`

The library of primitives is represented by an instance of the `Deltas` class. It's instantiated with `core` primitives (the initial primitives) and is enriched by `invented` abstractions

a 2d matrix is of type `grid` and a 3d matrix (a z-concatenation of grids) is of type `mat`.

monomorphic, string-based type system. No polymorphism, so each arrow over grids/pairs/stacks needs its own typed composer. So we need

- `pair_gg` for pairs (`grid,grid`)
- `fn_g_p` for pair producers `grid -> (grid, grid)`
- `fn_p_g` for pair consumers (`grid, grid`) `-> grid`
- `fn_p_p` for grid pair endomorphism (`grid, grid`) `-> (grid, grid)`
- `fn_g_c` for functions that read a coordinate of a grid `grid -> coord`
- `fn_gc_c` for functions that impose a coordinate on a channel (`grid, coord`) `-> grid`

The input of a trajectory task is the initial grid of a matrix, so its input is of type `grid`. a transition function `fn` is of type `grid -> grid`, since for any grid at timestep i it gives the next grid that follows at timestep $i + 1$. So when the enumerator is solving a trajectory task, only an `fn`-typed program is a valid solution since the interpreter for trajectory tasks (`unfold`) takes an `fn`. Then `unfold` takes the initial grid of type `grid`, the transition function of type `fn`, and the duration of the task `int`, and returns a 3d matrix `mat`, which can be compared to the original task matrix.

The input of a template task is a tuple of type (`grid,grid`). The goal of the task is to find a *commit policy* that answers the question “what should I do with the template?” so when the enumerator is solving for template tasks, only `fn_p_g`

3.5 Primitives

3.5.1 core primitives

every run’s library starts with core primitives used to construct the transformation functions. `step :: cellvalue, dir -> fn` moves the value one step in a specified direction, chosen among `left :: dir`, `up :: dir`, etc. `optimize :: util, cellvalue -> fn` moves the value one greedy (BFS-optimal) step towards improving its utility, chosen among `distance :: cellvalue -> util` or `neg_dist :: cellvalue -> util`.

3.5.2 atomic (phase 1)

- `fork :: fn -> (grid, grid) -> fn`
- `sync_to_world :: cellvalue -> (grid, grid) -> grid`

3.5.3 decomposed (phase 2)

The product-category combinator family in our type system is

```
dup_g      : grid -> pair_gg
mapsnd     : fn   -> fn_p_p
compose_gp : fn_g_p, fn_p_p -> fn_g_p
pipe_gpg   : fn_g_p, fn_p_g -> fn
```

which are just the pair-grounded instantiation of universal categorical combinators

- `dup : a -> pair a a` (the diagonal Δ)
- `mapsnd : (b -> c) -> pair a b -> pair a c` (the bifunctor ‘second’)
- `compose : (b -> c) -> (a -> b) -> (a -> c)`

The decomposition is polymorphic in principle. Our monomorphic primitives are the ground instances at `a = grid`. So belief *can* be assembled from standard categorical combinators.

so if we had polymorphism we could collapse the above family into one `compose`.

3.5.4 arity (phase 3)

3.6 ECD architecture

ECD is a wake-sleep library-learning loop inspired by the DreamCoder [Ell+20]. Each iteration of the loop consists of an enumeration phase, a compression phase, and a dreaming phase. In enumeration, the system generates candidate programs to solve tasks

3.6.1 Enumeration

Candidate programs are enumerated in order from most to least probable, i.e. from lowest to highest cost. Rather than setting out to synthesize a solution program for each particular task, the enumerator generates programs arbitrarily from the grammar. Once a well-formed program is completed (i.e. it has no holes), it is checked against the task corpus to see if it solves a task.

To illustrate, we will show how the enumerator finds (really stumbles upon) a solution program to a certain task. Say the task to solve depicts the value 3 rising in each successive grid, so the target solution is the program `(step 3 up)`. Its head is `step :: cellvalue, direction -> fn` and its leaves are `3 :: cellvalue` and `up :: direction`. A program’s cost is given by the sum of the costs

of its constituent primitives. Say Q is the initial flat prior, the type-uniform distribution, so each primitive’s cost is the negative natural log of the number of symbols in the grammar that share its type. For example, there are $N = 4$ symbols of type `direction` so each gets $p = \frac{1}{N} = \frac{1}{4}$. By Shannon, we have $\text{cost}(\text{up}) = -\ln p = -\ln \frac{1}{4} = \ln 4 \approx 1.386$ nats. By the same process, `step`’s cost is $\ln 8 \approx 2.1$ nats and `3`’s cost is $\ln 10 \approx 2.3$. By logarithmic additivity, to find the program’s total cost we simply sum the costs of its constituent primitives which yields ≈ 5.8 nats for `(step 3 up)`.

Program search proceeds by budget window to ensure that cheaper programs are enumerated first. For `(step 3 up)`, the first two windows are skipped by pruning. We start enumerating in window $[0, 2)$, but since `step`’s cost is 2.1 nats it exceeds the budget, so it cannot be chosen as the root node of the program tree and we proceed to budget window $[2, 4)$. After paying 2.1 nats for `step`, only 1.9 nats remain to fill the two holes in the partial program tree. `step`’s tail types (arguments) are `cellvalue` and `step`. But any primitive of type `cellvalue` costs 2.3 nats, so `step`’s first argument can’t be placed. So trees with a `step` node are pruned, since they would never complete within the second budget window. Search is cheapest-first, since more complex programs only become affordable once the window is wide enough.

At budget window $[4, 6)$ the ceiling is finally sufficiently wide to admit all three symbols. The tree is assembled incrementally: each enumerated subtree is appended to a copy of its parent node, and a callback fills the remaining argument slots by the same recursion (so programs of any arity or nesting depth are handled by the same procedure). Since enumeration is type-directed, each argument slot draws candidates only from symbols of the matching type, so no ill-typed partial tree is ever constructed. `(step 3 up)` completes at its total cost of ≈ 5.8 nats. Since the tree has no more available holes, the callback fires: the finished program is run on each task’s input grids and its output is compared against the target. If `(step 3 up)` is the first solution found for the task, it is stored as the task’s solution. Because the windows are tried in increasing order of cost, the first solution the search returns is (up to ties within a window) the shortest program that solves the task, which is its minimum-description-length explanation under the current grammar.

`Delta.length` is the node count of an expression tree. Every operator and leaf adds to the count, and an invented abstraction used as a token adds one to the count too. The body of the abstraction isn’t charged, since the components now count for just application. When the searcher needs to tiebreak between two programs that solve a task at the same cost, the program with the lower `length` wins since it has the lower minimum description length (since by symbol count it’s shorter).

`Delta.simplify` is a semantics-preserving rewrite that collapses spurious nesting a program can acquire during search. Two syntactically different programs can compute the identical trajectory, so it’s used to choose the canonical one. Sometimes one of these identical-trajectory programs has redundant `fork` wrappers whose `commit` re-applies a move the inner program has already made, or a `fork` whose `commit` merely reads back its own private channel (`fork(f, snd_gg) ≡ f`). This is just an artifact of enumeration, sometimes after Q -biasing the cheaper move is to nest `fork` unnecessarily. So `simplify` rewrites such variants to a canonical form so that the length tiebreak compares them fairly based on the intended semantics, so the stored solution is in fact the shortest one.

3.6.2 Compression

While Enumeration minimizes the cost of each program under a fixed grammar, Compression lets the system revise the grammar in light of the solutions found by enumeration. It identifies syntactic structure common across found solutions, and abstracts it as a new primitive to add to the library. The goal of compression is to minimize the joint cost of the library plus the programs written in it, given by:

$$\min_{\text{library}} \left(\text{DL}(\text{library}) + \sum_{\text{tasks}} \text{DL}(\text{program} \mid \text{library}) \right)$$

The choice of whether to abstract a given shared structure is non-trivial. Consider a program tree fragment that recurs across many solutions. Say it gets added to the library as a new primitive `fn_k`, so now each subsequent usage of the fragment collapses from a multi-node subtree to a single leaf, so it costs one token rather than the several that `fn_k` abbreviates. But the library is now larger, and since a symbol’s cost is $-\ln$ of the number of symbols sharing its type, adding one more `fn_k`-typed symbol raises the cost of every primitive of that type. The abstraction is worthwhile only when the fragment is large enough, and recurs often enough, that the tokens saved across the corpus outweigh the price of carrying one more symbol. So accidentally-shared structure is never abstracted, while shared structural motifs do.

In DreamCoder [Ell+20] the compression step is bottom-up: for each found program it enumerates the exponential set of program refactorings (stored as version spaces) and intersects them to grow abstractions up from reoccurring subexpressions. Rather than this bottom-up approach, we delegate the search for abstractions to the top-down Stitch [Bow+23] library-learning system. Instead of enumerating refactorings of every program, stitch searches the space of abstractions directly. So stitch doesn’t have to materialize every rewrite.

To create an abstraction, stitch starts with a tree that consists of just a root node `??`, an unexpanded hole that the search still needs to refine. At the root `??`, the abstraction *matches* every subexpression, since it’s the most general abstraction possible: any subexpression can be trivially “rewritten”

. As the tree is built up and the search is refined, the set of possible match locations shrinks.

A node of the stitch abstraction search tree is a partial abstraction. It consists of some combination of primitive operations (like `step`) and argument holes `#i`, and at least one unexpanded hole `??` that the search still needs to refine.

At each step it commits part of the abstraction’s body to a concrete operator or splits a subtree, branching downward toward more specific abstractions.

upper bound for a partial abstraction $A_{??}$ [Bow+23]

$$U_{\text{upperbound}}(A_{??}) = \sum_{e \in \text{matches}(A_{??})} \text{size}(e)$$

goal of compression is to minimize the size (cost) of program corpus after rewrite. Compressive utility function:

write about stitch compressive utility function

Each compression phase consists of a fixed number of stitch iterations, determined by the `iterations` parameter. Each iteration searches for the single most compressive abstraction, rewrites the corpus in terms of it, and repeats, so later abstractions may be built out of earlier ones. The `max_arity` parameter bounds how many holes (`#0`, `#1`, ...) an abstraction may carry.

Before passing the found solutions to stitch for abstraction, we first need to fully expand any existing primitives in those solutions. For this we implement `normalize`, the complementary operation to abstraction. It fully inlines every invented `fn_k` back to the bare primitive alphabet, recursively substituting each abstraction’s argument holes with its actual arguments until nothing but core primitives remain.

We need to normalize because stitch’s top-down compression can only abstract over structure it can see, and an opaque `fn_k` token has no visible interior. So any abstraction that would need to reach inside `fn_k`, or span its boundary and surrounding context, is simply invisible to the search.

the compression step must be handed programs in a common vocabulary — feeding it already-compressed programs, in which a prior round’s `fn_0` appears as an opaque token, would let those tokens collide with the fresh abstractions the compressor invents under the same names. Normalizing before compression guarantees the compressor sees only primitives and the structure it discovers is genuinely its own. Renaming it to `inv_0` avoids the lexical clash but leaves it just as opaque; stitch still can’t see the `(compose (wall_at ...)) (optimize ...)` sitting inside it. Inlining flattens the token back to that primitive structure, and only then can the compressor find abstractions that cut across the old boundaries.

The best syntactic fragment to abstract from round N ’s solution may carve the primitives differently than round $N + 1$ ’s. For example,

So we can refactor abstractions that were previously ideal but are no longer ideal

enumeration is by root types so `fn` and `fn_p_g` are searched separately. but compression is joint over root types, their solutions are pooled into a single stitch call. so an abstraction for belief has to fight over the space of all solved programs, including those from the other root type. so it really is that much more efficient. This preempts the attack that by separating the search to different root types we’re biasing abstraction creation towards the belief tasks since they make up a higher percentage of `fn`-rooted tasks rather than `fn_p_g`-rooted. the same objective that could have paid for belief’s structure is free to spend its budget on the overlay, registration, and obstacle families instead, but it ends up going to the belief families because those abstractions compress so much structure.

What it returns is a set of abstractions together with the corpus rewritten to use them. We infer each abstraction’s hole type and register the abstraction as a new `fn_k` in the library `D`.

The rewritten corpus stitch returns then becomes the training data for the dreaming phase and the starting library for the next round, in which programs a level deeper — belief compounds too expensive to enumerate from bare primitives — have become reachable because their shared core now costs a single token.

3.6.3 Dreaming

the recognition model

replays and fantasies

Chapter 4

Results and Discussion

belief/theory-of-mind is a discoverable compound under MDL rather than a built-in primitive

4.1 Found abstractions

4.1.1 atomic primitives

round 1 enumeration solves obstacle tasks

```
[ 1037] caught (compose (optimize (neg_dist 5) 4) (wall_at c1 c0))
```

and simple a few (7) simple false belief tasks

```
[102603] caught (fork (compose (wall_at c1 c2) (optimize (neg_dist 1) 7)) (sync_to_world 7))
```

so at the end of the round it abstracts a function for placing a wall and navigating around it

```
fn_0: (compose (wall_at #3 #2) (optimize (neg_dist #1) #0)) [cellvalue, cellvalue, coord, coord] -> fn
```

and uses `fn_0` to abstract the belief signature `fork(derive, commit)`: on the model grid, run `fn_0` as the `derive`, then run `sync_to_world` as the `commit` using the same `#0` (agent value) as passed to `fn_0`. So run the calculation for `#0` based on the modified grid, and then commit the conclusion to the real world.

```
fn_3: (fork (fn_0 #0 #3 #2 #1) (sync_to_world #0)) [cellvalue, coord, coord, cellvalue] -> fn
```

round 2 enumeration solves the rest of the simple false belief tasks using `fn_3`, requiring just around 1% of the original enumeration cost. So it can successfully use the learned abstraction to generalize. It's much easier to solve the false belief tasks when it already has a function and just needs to fill in the values, rather than needing to rediscover the entire structure of belief again

```
[ 1053] caught (fn_3 4 c3 c2 5)
```

at the end of the round it abstracts the belief signature again, which was `fn_3` in round 1

```
fn_6: (fork (compose (wall_at #3 #2) (optimize (neg_dist #1) #0)) (sync_to_world #0)) [cellvalue, cellvalue, coord, coord] -> fn
```

and it abstracts simple desire

```
fn_7: (optimize (neg_dist #0)) [cellvalue, cellvalue] -> fn
```

so then in **round 3** it can use those to solve witness-belief tasks

```
[ 11634] caught (compose (fn_6 5 8 c3 c3) (fn_7 2 1))
```

final abstractions: belief constructor

```
fn_6 [cellvalue, cellvalue, coord, coord] -> fn
body: (fork (compose (wall_at $3 $2) (optimize (neg_dist $1) $0)) (sync_to_world $0))
```

4.1.2 decomposed primitives

final abstractions

```
fn_6 [cellvalue, cellvalue, coord, coord] -> fn
body: (pipe_gpg (compose_gp dup (mapsnd (compose (wall_at $3 $2) (optimize (neg_dist $1) $0)))) sync_all)
  *** AGENT TYPE CONSTRUCTOR (belief – degenerate scope-complement commit) ***

fn_7 [cellvalue, cellvalue] -> fn
body: (optimize (neg_dist $0) $1)
  (desire fragment)

fn_8 [cellvalue, cellvalue, coord, coord] -> fn
body: (compose (wall_at $3 $2) (optimize (neg_dist $1) $0))
  (obstacle/belief policy: stamp wall ▶ navigate – the shared derive)

fn_9 [coord, coord, cellvalue, cellvalue] -> fn
body: (compose (optimize (neg_dist $3) $2) (wall_at $1 $0))
  (obstacle/belief policy: stamp wall ▶ navigate – the shared derive)

fn_10 [coord, cellvalue, cellvalue] -> fn
body: (pipe_gpg (compose_gp dup (mapsnd (compose (wall_at c2 $0) (optimize (neg_dist $1) $2)))) sync_all)
  *** AGENT TYPE CONSTRUCTOR (belief – degenerate scope-complement commit) ***

fn_11 [fn, fn_p_g] -> fn
body: (pipe_gpg (compose_gp dup (mapsnd $0)) $1)
```

4.2 The structure of theory of mind

explain the structure of ToM, of the agency signature, of belief attribution

belief is a function that takes a proposition and an agent

```
fn_0 = (fork (compose (wall_at $3 $2) (optimize (neg_dist $1) $0))
  (sync_to_world $0))
```

in which the agent value `$0` appears twice. once in the policy that plans over the private, phantom-walled grid built by `derive`, and again in the `sync_to_world` commit that writes the result back to the world.

Nothing in the grammar forced those two argument slots to be filled by the same value. The fact that they collapse into one shared hole is the structural signature of agency, of an agent acting

on its own model of the world. This signature is discovered by the description-length objective, it’s not stipulated by a primitive.

4.3 Benefits of concept learning

The guiding motivation of this project is that an intensional theory, attributing a belief to an agent, is the most compressed representation of an agent’s behavior. This is obvious, since the agent navigates based on its intensional representation, so a theory that accounts for this will better represent the underlying data generating process. It’s certainly possible to describe an agent’s movement without attributing a belief to it, but it would be a laborious explanation that accounts for each step in the sequence. And of coruse it doesn’t generalize well, it overfits to a specific agent’s movement in a particular scene.

To illustrate this we show that no non-intensional solution is as short as one which stipulates belief attribution. As described in chapter 3 the enumerator processes candidate programs in $-\log p$ order and saves just the first (cheapest) solution it finds, call it *found*. For false belief tasks should be a program that attributes a belief to an agent. Since a false belief task can also be solved by a purely extensional program, call any such program a *rival* since it’s an extensional rival to the intensional explanation.

For each rival, we generate a *margin* score by taking $margin = DL(rival) - DL(found)$, where DL is the description length (how many nats it takes to write program p under library L). So for any rival, if $margin > 0$ then the intensional program is shorter than the extensional, otherwise an extensional program is at least as short. We plot the empirical cumulative distribution over the margin values for all rivals, so the x -axis shows possible margin values and the y -axis shows the cumulative fraction of rival programs with $margin \leq x$. Since description length depends on the given library we plot two ECDFs: dotted for programs written in the base library, and solid for programs written in the final library (enriched with abstractions found through five ECD rounds).

We first give the plot for the higher level DSL in which `fork` and `sync` are given as atomic primitives.

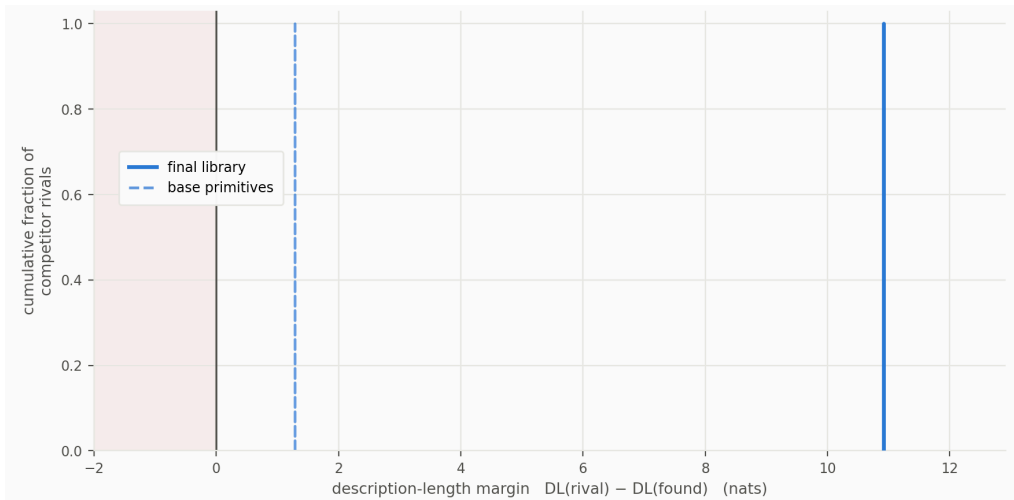


Figure 1: ECDF of MDL margin over rival programs (atomic DSL)

Both curves sit entirely to the right of $x = 0$, so for each of the 24 false belief tasks, the *found* (intensional) program is a shorter description length than any *rival* (extensional) program. In the base (dashed) library, rival programs cost only ≈ 1.3 nats more than the found program. But the margin balloons to ≈ 10.9 in the final (solid) library: once the `fork(derive, commit)` abstraction is added, the enumerator can immediately generate programs that posit an agent’s intensional state. Without such a learned abstraction, an intensional program is only slightly less verbose than an extensional one. But with the abstraction, intensional programs are vastly more compressed than extensional ones.

This effect is even more pronounced in the plot for the lower-level DSL, in which `fork` and `sync` are decomposed to their product category components.

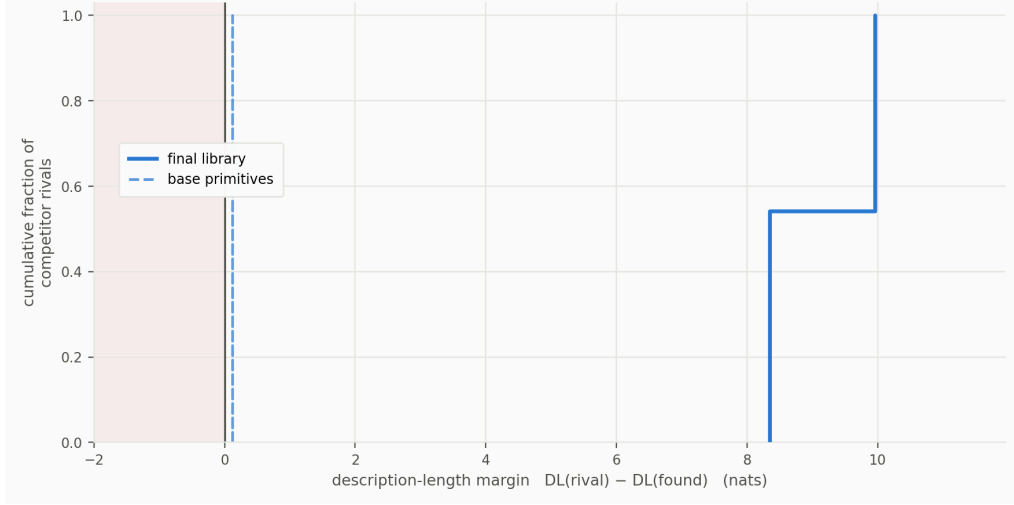


Figure 2: ECDF of MDL margin over rival programs (decomposed DSL)

Under the base primitive library, the mental reading is barely shorter than the physical one (≈ 0.1 nats). Since the intensional machinery of forking and syncing needs to be constructed from lower-level components, length of intensional programs end up virtually tied with that of the verbose extensional programs. But once the belief abstraction `fn_0` is learned and added to the library, the margin grows to $\approx +8.4$ nats. So mental programs go from being just as bad as non-mental ones to being vastly better, through learning a fitting abstraction. Note that the jump to a $\approx +9.96$ nats margin for the last 11 tasks is due to a further specialized `fn_4 = (fn_0 ... c2)` that hard-codes a phantom-wall at a particular coordinate `c2` so such programs are one coordinate cheaper. But the rival cost is constant across both, so the entire step is a property of how tightly the library compressed the intensional side.

We can also see this effect when comparing description lengths of different families of tasks: belief, desire, physics, and world. To plot, we fix each corpus (task type) and calculate each corpus’s description length given a round as such:

$$\text{DL}(C|r) = \sum_{c \in C} \text{DL}(p_c|L_r) + \text{DL}(L_r)$$

where C is a task corpus (e.g. belief tasks), c is an instance of a task in that corpus, r is an ECD round, and p_c is the found (minimal) program that solves task c . So calculate the description length of a corpus at a given round, for each task in the corpus $c \in C$ we sum the DL of each program p_c

that solves it, given under that round’s library L_r . And we add the description length of the library L_r itself, a structure term to account for the cost of a more complicated library.

Under the base library, total description length (black line) of the entire task corpus (every C) is 3209 nats. After four ECD rounds it drops to 2076 nats, an 1134 nat compression. We see the biggest drop in the belief corpus, once the system learns to efficiently represent intentional agents using the `fork(derive, commit)` abstraction. Once that constructor is in the library, every belief task rewrites through it and the corpus gets dramatically shorter at once.

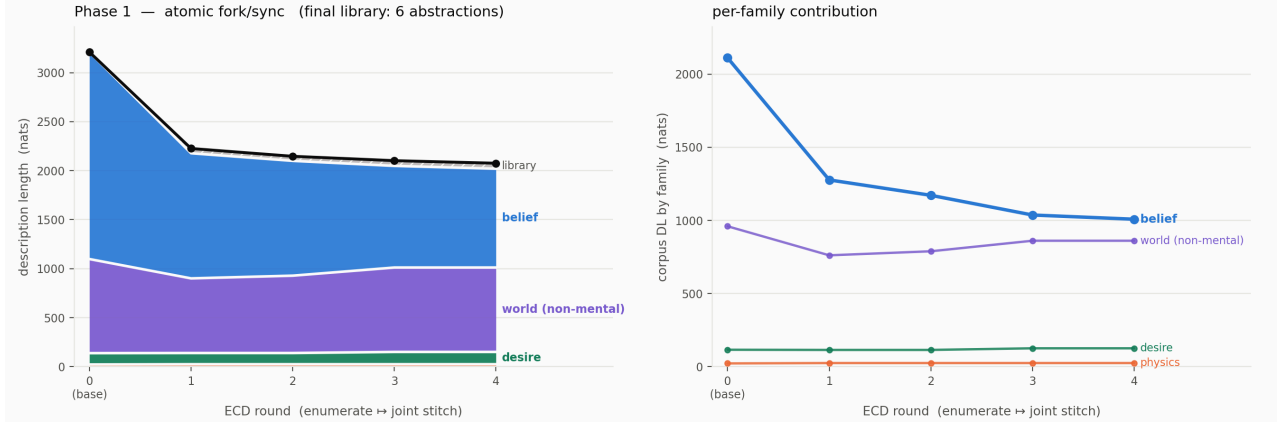


Figure 3: Description length per task family (atomic primitives)

Unlike the belief corpus, the description length of the other (non-mental) tasks remains mostly stable as new abstractions are learned. They’re solved just as easily under the base primitives as they are under the final library (a bit worse actually since the library is more complex). If the initial primitives *did* encode mental notions, if there were a native ToMM, we’d see similar results for the belief tasks: they’d be solved just as easily from the start. But since iterated abstraction yields a significant improvement, it’s clear that the initial primitives are not mental but that the ECD loop enriches the library with abstractions that *do* encode mental concepts (the concept of belief attribution).

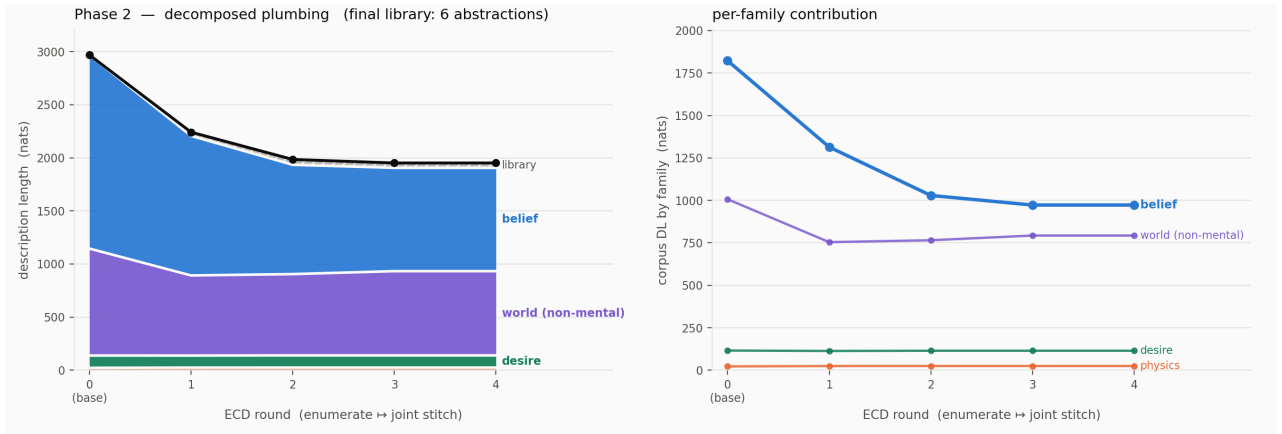


Figure 4: Description length per task family (decomposed primitives)

Belief tasks are reached late and cheaply. They only become solvable once the ECD loop has abstracted the reusable parts, and the moment that abstraction is added to the library, belief tasks flip from timing out to being solved in seconds. See the following plots of cumulative task solutions per round, and per-task solve time.

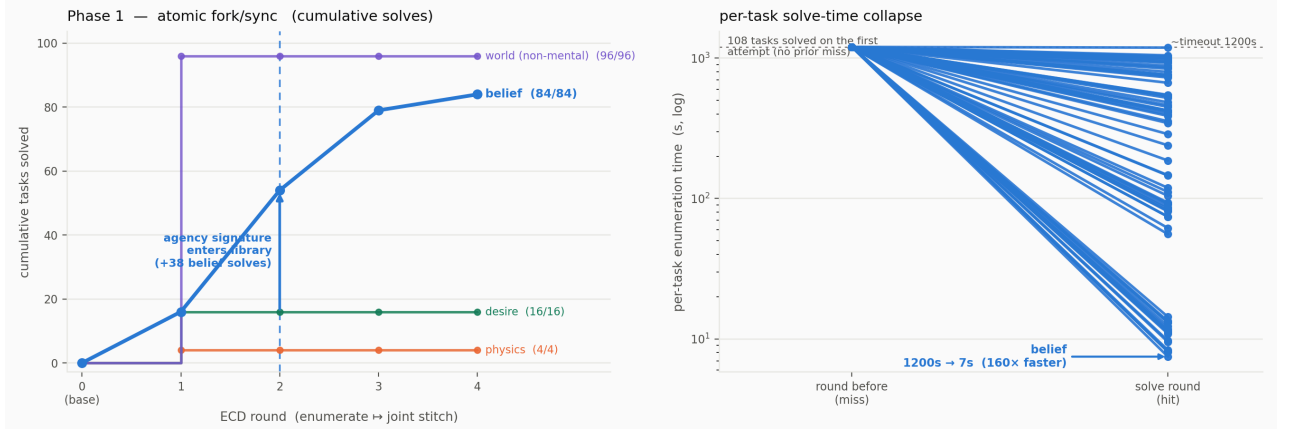


Figure 5: Cumulative solves by task family (atomic primitives)

Tasks other than belief get solved in the first round, they don't need abstractions. But belief can't be enumerated from base primitives, so it exhibits an S-curve. There are very few solves early on, since each solution must be laboriously composed of deep nested primitives. Then there's a sharp step up when the stitch finally abstracts the agency signature, the shared-model structure belief reuses. Then the learned abstraction is used to solve the remaining belief tasks, so the solved tasks plateau for the last round.

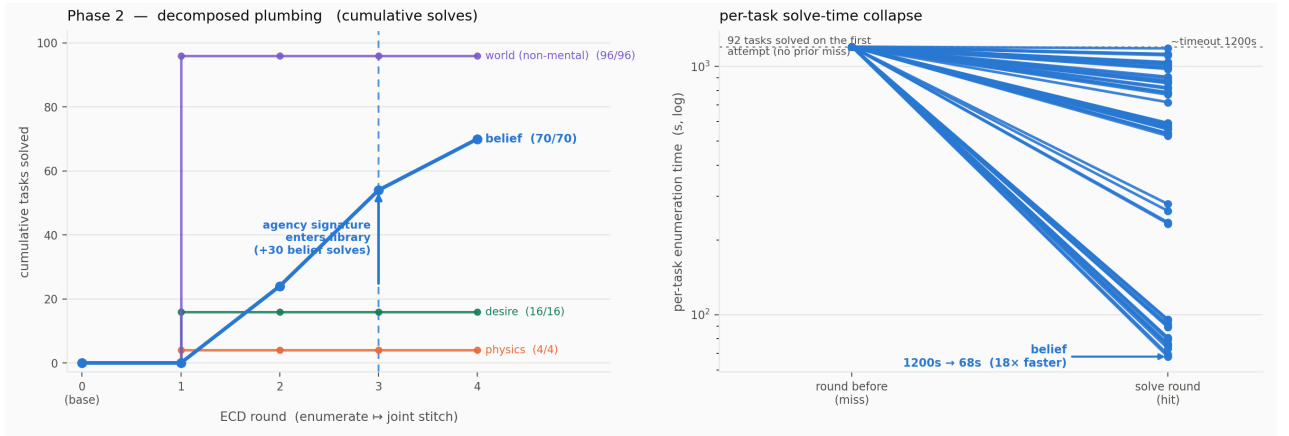


Figure 6: Cumulative solves by task family (decomposed primitives)

Starting with the atomic library, the abstraction jump occurs at round 2 where we abruptly see +38 belief solves. If we start with the decomposed library, the abstraction takes an extra round to assemble so the step is at round 3 where we see +30 belief solves.

Plot 2: per-task solve time, round n vs round $n+1$, as a paired slope-graph or log-scale scatter — the 1100 s $\rightarrow$ 10 s collapse is currently buried in log text.

4.3.1 Per-variant belief coverage + combo

Plot 1: Bar chart of solved/total for wall/witness/goal/dual

Plot 2: heatmap of $(gv, av) \times$ abstraction-used showing fn_8 tiling every cell.

Rows are the (goal value, agent value) combinations in a task, columns are the invented abstractions added to the library. The count indicates how many solutions use that abstraction.

INSERT FIGURE

So the belief abstraction generalizes across the (goal, agent) combinations it was trained on.

4.3.2 Behavioral probe: Passing the false-belief test

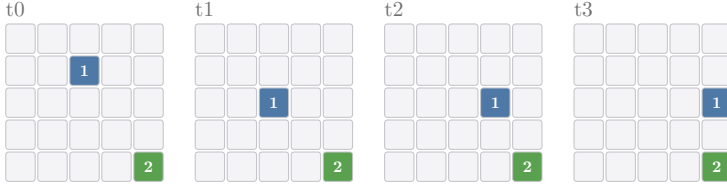
The analyses so far are about description length, showing that the intensional program is the shorter account of the data thanks to the abstraction that makes it easier to express more complex programs. We now show that the system passes the false belief test, reproducing the classic Sally-Anne false-belief setup [WP]. The test simply asks where the system predicts the agent will look. An object is where the agent last saw it, but the agent’s belief about its location is now false; a child who attributes belief points to the believed location, while a younger child who cannot yet attribute belief points to the object’s true location.

We use the goal-displacement family of tasks for this. These tasks depict a value’s trajectory that changes direction 90°

Goal displacement — Sally-Anne ^[belief_goal]

```
(fork (seq (step gv d) (optimize (neg_dist gv) av)) (sync_to_world av))
```

av=1, gv=2, displaced_to=(2, 4), dirs=('up', 'up')



, because unlike the wall and witness families it lets the two readings come apart behaviorally. In a wall-belief scene the extensional rival posits a transient wall in the actual world rather than positing a wall in the agent’s private believed world. That rival reproduces the entire trajectory, which is why the MDL margin, not behavior, is what separates them there. But in a goal-displacement scene the agent walks to where it believes the goal is (one cell from the true goal, which never moves) so a program that does not represent the agent’s belief cannot reproduce the walk. The two readings predict the agent settling on different cells, which is the false-belief test.

The probe reuses the phase 1 run and re-stitches its solutions to rebuild exactly the library of abstractions it converged to (this is the same reconstruction that the MDL-margin experiment performs). Onto a held-out scene (drawn from a fresh seed that never entered the run, and checked to be absent from the training corpus) we instantiate the discovered belief compound (the `fork(derive, commit)` abstraction the library reprices belief through) and, as its foil, the shortest program expressible in the same library. Figure 7 shows one such scene.

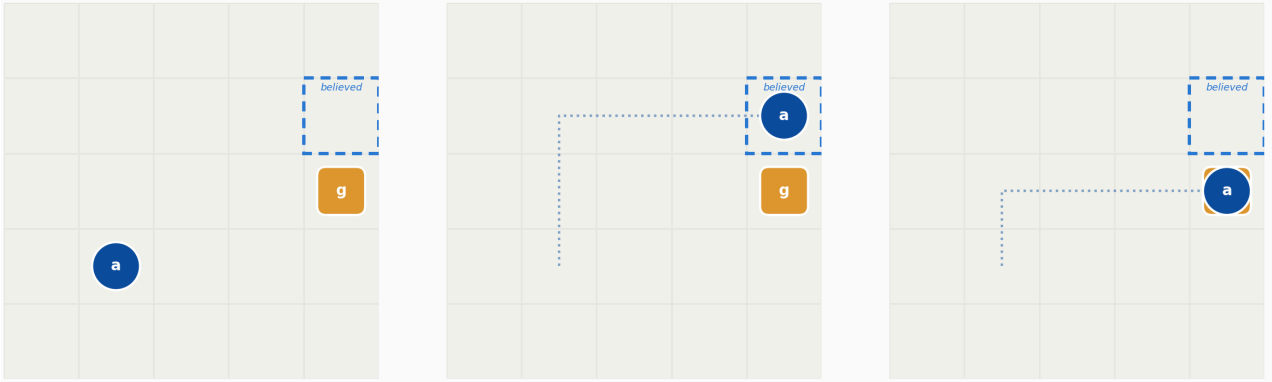


Figure 7: Behavioral probe on a held-out Sally-Anne scene. The learned belief compound sends the agent to the believed cell (passes); the shortest non-mental program under the same library sends it to the true goal (the naive answer).

With `fn_1` as the abstraction

```
(compose
  (optimize (neg_dist #1) #0)
  #2)
```

The learner equipped with the belief compound writes the program

```
(fork
  (fn_1 1 2 (step 2 down))
  (sync_to_world 1))
```

Which is normalized as

```
(fork
  (compose
    (optimize (neg_dist 2) 1)
    (step 2 down))
  (sync_to_world 1))
```

the discovered `fork(derive, commit)` abstraction applied to a `derive` that privately displaces the goal and seeks it, committing only the agent’s own move back to the world via `sync_to_world`. Its final frame puts the agent on the believed cell, one step past the true goal. It searches where the agent thinks the goal is. The learner restricted to the non-mental fragment writes the shortest thing that fragment can say, plain desire `(optimize (neg_dist 2) 1)`, and its agent walks straight to the true goal, where the goal really is. Across all 24 held-out scenes the belief compound lands on the believed cell every time, the shortest non-mental program on the true cell every time.

Note that here the non-mental program is shorter than the intensional one (7.9 nats against 20.5), which is the opposite of the MDL-margin result, and this is the point. Where the transient-wall rival reproduces the scene, it does so only at a longer description length, so MDL rules it out. Where the pure-desire rival is cheaper, it does so only by predicting the wrong behavior, so the data rule it out. There is no non-mental program that is both as short and as accurate as the belief

compound. The naive reading is available and parsimonious but it fails the test. A learner that has compressed its experience into a belief compound passes the false-belief task. a learner confined to the non-mental fragment fails by predicting the true location rather than the believed one. The abstraction the loop discovers for the sake of compression what licenses the mentalistic prediction.

4.4 Ablation experiments

4.4.1 Silo vs joint stitch

compress belief solutions alone vs pooled with the non-mental corpus. Is the constructor still found, and at what DL?

Plot: library-DL and constructor-invention bar pairs.

4.4.2 Corpus-mix dose-response

vary the non-mental fraction (0%, 25%, 50%, ...); measure constructor invention and belief solve rate. Shows the non-belief tasks are the scaffolding that gets us to abstract belief

Plot: solve rate / invention frequency vs mix.

4.4.3 Budget dose-response

–t-fn at 600/1200/2400 s; belief solve rate per variant. Supports argument that misses are budget-bound, not due to representational limitation

Plot: solve rate vs budget, per variant (dual should be the steepest riser).

4.4.4 Dreaming ablation

–no-dream vs dreamed Q: cumulative solves vs wall-clock. Round 3-vs-4 in the log already hints the dreamed Q matters.

plot a curve that shows Q bias contribution

4.4.5 curriculum ablation

remove certain tasks, show that low-level abstractions don't get abstracted and then that prevents us from reaching high level abstractions

4.4.6 Seed robustness

5 corpus seeds × the main run; report every verdict metric as k/5 with solve-rate error bars

4.5 Discussion and rebuttals

question: to what extent is the ToM already present in the input data and DSL and structure

potential argument: `fork` and `sync_to_world` are just decomposed `believes`. By your definition you've been using from the literature, ToM/mentalizing is to be able to attribute a belief to an agent. A belief is an intensional state, a grid representation that may differ from the world. So `fork(derive, commit)` is already belief attribution, you've just broken it apart into components and now they're put back together.

Responses:

1. `fork` and `sync` have non-mental uses, and they're often used independently of each other to solve other things, so they're general-purpose

2. there are also lots of other primitives that are the duals (cube axis), and those don't get selected cube of symmetric complements, and general pair interface (**overlay**, **registration**) that stops **fork/sync** from being a disguised **believe**.

potential argument: the **fork** primitive smuggles in ToM because creating a copy of the world grid is a representational capacity

response: ToM isn't just the representational capacity. creating a copy of the world that differs from it is just counterfactual reasoning, the what-if ability. it's super domain-general. infants have been shown to have it, disjunctive reasoning. The ToM is an abstraction that *uses* this domain-general counterfactual representational capacity to attribute a counterfactual world to a particular agent - that this modified world is *that agent's* world.

potential argument: **optimize** is basically ToM because BFS navigation requires the agent to recognize a

out of the full $2 \times 2 \times \dots$ cube of channel-direction / channel-target / z-order / scope variants, joint MDL still selects exactly the read-model $\rightarrow$ write-world, single-av corner for belief, and leaves the symmetric variants attached to their non-mental tasks.

pair apparatus was available to every task, but recruited by only the false belief ones. Those are the trajectories that can't be accounted for by only a single grid, so the search pays + 10 nats for the second grid only when nothing cheaper explains the data. Representationalist's criterion: posit the hidden world only if it pays for itself, only if its necessary. Availability is not use. The inference "this agent needs a belief" is made, per task, by cost-driven model selection.

The (**grid,grid**) and arrow types aren't smuggled-in ToM, they're just the logical form that ToM requires. Intensionality is the coexistence of world and model. Any substrate expressive enough to even state "act on the world via a transformed model of it" must be able to hold two representations at once. And "two things held at once" is a product, in some guise (a pair, two registers, two tape regions, two variables). You cannot discover ToM in a substrate that can't hold two representations [for the same reason you can't discover addition in a language with no notion of "two numbers." The pair is to theory of mind what the integers are to arithmetic.] And the pair is symmetric and content-free: **dup** makes (**w**, **w**). There's nothing in it that says "the second one is the model, the believed world." The asymmetry that makes one copy real and the other believed is: transform the second (**mapsnd**, not **mapfst**), commit from model to world (**sync_to_world**). This asymmetry is entirely in the composition, which is the discovered part. The medium is neutral, it could go any way, as seen in the other tasks which do use those other primitives and wire them in other ways. But the system is able to discover this particular composition of these particular primitives to solve belief tasks, via programs that constitute a theory of mind.

Sure you can keep going. Decompose **dup** from a flat memory/tape with **allocate/read/write**, so "hold a second world-model" becomes a discovered pattern of buffer use. But be clear-eyed: that relocates the regress, it doesn't end it. "Allocate a buffer" is then the primitive, and it's no more or less mental than "form a pair" — the capacity-for-plurality is presupposed by intensionality at every level, because it's constitutive of intensionality. There is no substrate that discovers ToM

“from nothing”; there’s only substrates whose primitives are, or aren’t, individually non-mental and general.

generality by reuse is what shows it

1. the same combinators used to create the belief abstraction are used in non-mental tasks
 2. the library is also initialized with symmetric primitives to the ones used in the belief abstraction
- so it’s a domain-general library that the search happened to recruit for belief.

Chapter 5

Further work

5.1 Comparison to LLMs

I'm doing search in program-dynamics space, not token space. Synthesizing programs is logical and semantic discovery, unlike token prediction. ECD enumerates typed program trees by probability and runs them on grids; there is no surface-syntax statistics anywhere.

5.2 Polymorphism

We use a simply-typed monomorphic type system. For our purposes, types are really used only to prune enumeration. They constrain which compositions are legal, not what the program must prove. DreamCoder uses polymorphic types, so unification can rule out ill-typed branches early. But since our DSL has only ~ 30 primitives our bottleneck isn't branching factor, so monomorphic dict lookup is sufficient.

With monomorphic types, a stitch abstraction discovered at type `grid` can only be reused at `grid`. With polymorphism, an abstraction abstracted over $\forall a$ can be reused across instantiations of any type a . Phase 3 hand-engineers this with `the cons/nil` stack leaving arity as a free parameter. If we had polymorphism it would just fall out of the type discipline.

To do something like that, we'd replace bare string types with a small ADT:

- `ground ('con', 'grid', [])`
- `variable ('var', 'a')`
- `applied ('con', 'pair', [t1, t2])`
- `arrows (args, ret)`

We'd implement Robinson unification

5.3 task sets

currently, to solve a task the enumerator needs to find a program that just generates that one sequence of grids. this is quite limiting from a task design standpoint, it makes it hard to design complicated tasks. it's why we need so many explainability filters.

instead, we could have a task be a handful of trajectories (three or four maybe), and then the goal is once again to find one program that explains them all. then we could have, let's say, two of those trajectories that can be explained by simple goal navigation, but one of the trajectories can only be explained by false belief. So the most general program, the simplest program, that fits all of them is one that posits the false belief.

e.g. in three scenes the agent has goal 3, but in

5.4 persistent models and interaction

currently an attributed grid state, a belief, exists only for that one frame, since we're just working with grid to grid transition functions. so do something where the agent actually has a persistent memory of its modified grid

and something where the agent can actually react to its environment, can write to the private model and read from it, can navigate based on obstacles, etc

5.5 breaking down optimize

this primitive packs in a lot, it's a full bfs search. so try to rediscover that. this requires a persistent model (to keep track of visited cells) and a and interactions with the environment, conditionals (if wall, move away)

5.6 probabilistic primitives

an agent that sometimes does this and sometimes that, etc

5.7 vs. real world

obviously this isn't how an infant learns theory of mind

I'm missing the *self*. I'm focusing on learning to distinguish between different entities in the domain, but not an infant learning to distinguish others from itself. For example, if I bite myself it hurts, but if I bite someone else it doesn't. maybe pain occurs only if I bite this but not that (extensional). or we have different states, different experiences, and sometimes someone else can be experiencing pain even when I'm not.

I'm missing the scale. I'm focusing on a narrow timescale. in real infants this would unfold over a long time scale.

Chapter 6

Appendix

- full list of tasks
- full list of primitives

Bibliography

- [MSS12] Margolis E, Samuels R, Stich SP, editors. The Oxford handbook of philosophy of cognitive science. Oxford New York: Oxford University Press; 2012. <https://doi.org/10.1093/oxfordhb/9780195309799.001.0001>.
- [WP] Wimmer H, Perner J. Beliefs about beliefs: Representation and constraining function of wrong beliefs in young children's understanding of deception n.d.
- [GCT24] Griffiths TL, Chater N, Tenenbaum J. Bayesian models of cognition: reverse engineering the mind. Cambridge, Massachusetts: The MIT Press; 2024.
- [Ten+11] Tenenbaum JB, Kemp C, Griffiths TL, Goodman ND. How to Grow a Mind: Statistics, Structure, and Abstraction. *Science* 2011;331:1279–85. <https://doi.org/10.1126/science.1192788>.
- [PTR11] Perfors A, Tenenbaum JB, Regier T. The learnability of abstract syntactic principles. *Cognition* 2011;118:306–38. <https://doi.org/10.1016/j.cognition.2010.11.001>.
- [Rul] Rule JS. Pre-cis of The child as hacker: Building more human-like models of learning n.d.
- [KPT07] Kemp C, Perfors A, Tenenbaum JB. Learning overhypotheses with hierarchical Bayesian models. *Developmental Science* 2007;10:307–21. <https://doi.org/10.1111/j.1467-7687.2007.00585.x>.
- [KT08] Kemp C, Tenenbaum JB. The discovery of structural form. *Proceedings of the National Academy of Sciences* 2008;105:10687–92. <https://doi.org/10.1073/pnas.0802631105>.
- [GUT11] Goodman ND, Ullman TD, Tenenbaum JB. Learning a theory of causality. *Psychological Review* 2011;118:110–9. <https://doi.org/10.1037/a0021336>.
- [Lak+16] Lake BM, Ullman TD, Tenenbaum JB, Gershman SJ. Building Machines That Learn and Think Like People 2016. <https://doi.org/10.48550/arXiv.1604.00289>.
- [BST09] Baker CL, Saxe R, Tenenbaum JB. Action understanding as inverse planning. *Cognition* 2009;113:329–49. <https://doi.org/10.1016/j.cognition.2009.07.005>.
- [Les94] Leslie AM. Pretending and believing: issues in the theory of ToMM. *Cognition* 1994;50:211–38. [https://doi.org/10.1016/0010-0277\(94\)90029-9](https://doi.org/10.1016/0010-0277(94)90029-9).
- [Ell+20] Ellis K, Wong C, Nye M, Sable-Meyer M, Cary L, Morales L, et al. DreamCoder: Growing generalizable, interpretable knowledge with wake-sleep Bayesian program learning 2020. <https://doi.org/10.48550/arXiv.2006.08381>.
- [Bow+23] Bowers M, Olausson TX, Wong L, Grand G, Tenenbaum JB, Ellis K, et al. Top-Down Synthesis for Library Learning. *Proceedings of the ACM on Programming Languages* 2023;7:1182–213. <https://doi.org/10.1145/3571234>.